

Tour Ceylon

Travel Dashboard Guide

A simple explanation of the trip planner — features, files, tech, and database.

Read this first

Travel is the trip-planning website. The user walks through steps like a wizard. Each step has its own screen (file). The browser talks to a Python server, and the server saves data in PostgreSQL. You do not need to memorize every file — this guide explains why each one exists.

1. What is the Travel dashboard?

Travel helps a tourist plan a Sri Lanka trip. The user picks places to visit, sets a budget, chooses hotels, gets a day-by-day plan, discovers local services, and downloads a PDF. There is also History (past trips), Profile (personal details), and an SOS button that opens the emergency app.

Normal order the user follows:

- Login — sign in
- Attractions — pick places and moods
- Budget — set money and split it
- Stay — pick hotels
- Itinerary — see day-by-day plan
- Discover — businesses, agencies, guides
- Export — download PDF

2. Why so many files?

The project splits work into small files on purpose. One big file would be hard to change and easy to break. Each file has one job:

- pages/ — one screen the user sees (Login, Budget, etc.)
- components/ — reusable UI pieces shared by pages (sidebar, photo)
- context/ — shared memory (who is logged in, current trip)
- api/ — code that calls the server
- backend routes/ — server endpoints (URLs the UI calls)
- backend models/ — one file per database table
- backend services/ — business logic (budget math, PDF, itinerary rules)

Think of it like a kitchen: plates (pages), utensils (components), recipe book (services), and pantry shelves (database tables). Separating them keeps cooking simple.

3. Tech stack — what we use and why

React 18	Frontend	Builds interactive screens without full page reloads
Vite	Frontend tool	Starts the app fast in development
React Router	Frontend	Each step has its own URL (e.g. /budget)
Axios	Frontend	Sends HTTP requests to the Flask API with the login token
Recharts	Frontend	Draws the budget pie chart
Flask	Backend	Python web server that answers /api/... requests
Flask-JWT-Extended	Backend	Creates a secure login token (JWT) after login
SQLAlchemy	Backend	Lets Python read/write Postgres tables as objects
PostgreSQL	Database	Stores users, trips, budgets, itineraries permanently
Google Sign-In (optional)	Auth	Login with Google instead of typing password

In development: the Travel UI runs on port **5180**. The API runs on port **5002**. Vite forwards /api calls to the API so the browser does not hit CORS problems.

4. How the app talks to the database

Browser (Travel UI) → saves JWT in localStorage → Axios calls /api/... → Flask route → SQLAlchemy model → PostgreSQL table.

- Login checks the users table and returns a JWT token.
- Every later request sends that token so the server knows who you are.
- TripContext keeps the current trip in browser memory (for smooth UI).
- PostgreSQL keeps the real saved trip (so History can reload it later).
- If you refresh mid-wizard, memory is cleared — reopen from History or start again.

5. Features explained clearly

5.1 Login

User signs in with email/password or Google. The server checks or creates a row in **users**, returns a JWT, and the app stores it. It may also link a Tourist SOS account so Profile and SOS share the same person.

Main files: `LoginPage.jsx`, `AuthContext.jsx`, `api/client.js` · *Backend:* `auth.py` · *Table:* `users`

5.2 Attractions

User sets trip days, picks up to 3 moods, selects attractions, then confirms. Confirm creates a trip in the database and stores tripId for the next steps.

Main files: `AttractionsPage.jsx`, `AttractionPhoto.jsx`, `TripContext.jsx` · *Tables:* `attractions`, `user_trip_input`

5.3 Budget

User enters total money and splits it across food, stay, shopping, transport. A pie chart shows the split. Values are saved to `budget_split`.

Main files: `BudgetPage.jsx` · *Backend:* `budget.py`, `budget_service.py` · *Table:* `budget_split`

5.4 Accommodation (Stay)

User picks one hotel per destination, filtered by the stay budget. Choices are saved on the trip row.

Main files: `AccommodationPage.jsx` · *Backend:* `accommodation.py`, `trip_plan_service.py` · *Table:* `accommodation`

5.5 Itinerary

The server builds a day-by-day plan from the selected places and hotels (rule-based from seed data, not inventing random places). User can rebuild with a different day start.

Main files: `ItineraryPage.jsx` · *Backend:* `itinerary.py`, `ai_service.py` · *Table:* `generated_itinerary`

5.6 Discover (Recommendations)

User browses businesses, travel agencies, and guides near trip destinations and can save favorites onto the trip.

Main files: `RecommendationsPage.jsx` · *Tables:* `business_directory`, `travel_agencies`, `tourist_guides`, `saved_references`

5.7 Export

User reviews the trip summary and downloads a PDF itinerary built by the server.

Main files: `ExportPage.jsx` · *Backend:* `pdf_service.py`

5.8 History / Profile / SOS

- History — list past trips and reopen one into TripContext.
- Profile — edit visitor details shared with the SOS app.
- SOS button — bridges the Travel login into the Tourist SOS app.

6. Frontend files — what each one does

Folder: **frontend/travel/src/**

main.jsx	Starts the React app and wraps Auth + Trip providers	App entry point
App.jsx	Defines all page routes and which need login	Navigation map
api/client.js	Talks to Travel APIs; attaches JWT on every call	One place for HTTP
api/sosClient.js	Links/bridges Tourist SOS session; opens SOS app	SOS handoff
context/AuthContext.jsx	Login, logout, Google; stores token	Shared login state
context/TripContext.jsx	Holds current trip choices in memory	Steps share one trip
components/Layout.jsx	Sidebar steps, progress bar, SOS/History/Profile	Common shell
components/AttractionPhoto.jsx	Shows attraction image with fallback	Broken URLs don't break UI
pages/LoginPage.jsx	Login and register screen	Public entry
pages/AttractionsPage.jsx	Moods + attractions → create trip	Wizard step 1
pages/BudgetPage.jsx	Budget total, split, pie chart	Wizard step 2
pages/AccommodationPage.jsx	Hotel picks per destination	Wizard step 3
pages/ItineraryPage.jsx	Generate and view day plan	Wizard step 4
pages/RecommendationsPage.jsx	Discover + save favorites	Wizard step 5
pages/ExportPage.jsx	Summary + PDF download	Wizard step 6
pages/HistoryPage.jsx	Past trips; reload into context	Reopen saved work
pages/ProfilePage.jsx	Visitor / SOS profile editor	Safety details
styles/global.css	Colors, layout, overall look	Consistent design
theme/sosProfile.js	Styles matching SOS profile	Same look across apps

7. Backend files — what each one does

Folder: **backend/** (Travel-related)

run.py	Starts Flask on port 5002	One command to run API
app/__init__.py	Creates app, DB, JWT, registers routes	App factory
app/config.py	Database URL, JWT secret, defaults	Central settings
routes/auth.py	Register, login, Google, me	User identity
routes/attractions.py	List attractions	Attraction API
routes/trip_input.py	Create / get / update trips	Trip API
routes/budget.py	Save / get budget split	Budget API
routes/accommodation.py	Hotels for a trip	Stay API
routes/itinerary.py	Generate, history, PDF	Itinerary API

routes/recommendations.py	Businesses, agencies, guides	Discover API
routes/saved_references.py	Save favorites on a trip	Favorites API
services/budget_service.py	Default % and money math	Keep math out of routes
services/trip_plan_service.py	Day plan + hotel matching	Planning rules
services/ai_service.py	Build itinerary from seed data	Itinerary builder
services/pdf_service.py	Create PDF file	Export
utils/trip_helpers.py	Check trip belongs to logged-in user	Security helper
database/schema.sql	Defines Travel tables	DB structure
scripts/travel/seed_data.py	Loads sample attractions/hotels	Demo data

8. Database tables

users	Travel accounts (email/password)
attractions	Places to visit (seeded list)
accommodation	Hotels (seeded list)
user_trip_input	The user's trip (core row)
budget_split	How budget is divided
generated_itinerary	Day-by-day plan JSON
business_directory	Local businesses
travel_agencies	Travel agencies
tourist_guides	Guides
saved_references	Favorites saved on a trip

Remember

UI screens collect choices → Flask saves them in PostgreSQL → itinerary and PDF are built from that saved trip → History reloads it. SOS is only a bridge from the top bar / profile.